

Runes

A recursive Ruby harness that rides on an MQTT fabric

The 30-second overview

Runes is an agentic harness: an LLM planner decomposes an intent into tool calls; trusted host tools (`write_file` / `read_file` / `run_command`) execute them inside a confined workspace; untrusted code runs in a `ruby.wasm` sandbox (fuel-metered, wall-clock-bounded). Interactive modes shape intent into artifacts: `/goal` → Epic, `/plan` → Mission (ordered todos + JSON sidecar), `/build` → sequential execution with a fail-closed QA verifier per todo.

Everything — prompts, plans, tool calls, results, agent presence, progress — travels as messages on one MQTT bus. That single decision is what makes the harness simple, observable, and fast to extend.

Why MQTT — the design bet

- **Broadcast by nature.** `runes/prompts` is a broadcast topic: every dispatcher hears every intent. Coordination (who executes?) is layered on top with a deterministic claim/lease protocol — no coordinator service, no queue infrastructure, no leader election library.
- **Decoupled fleet.** Agents publish and subscribe; nobody holds references to anybody else. Adding a second, third, tenth agent is zero code — they appear on the bus and immediately share work.
- **Topics are the addressing scheme.** `runes/agents/id>/card` · `runes/prompts/req>/progress` · `runes/sessions/lease>/claim` · `runes/tools/tool>/request` — a hierarchical naming scheme that doubles as an addressing and capability system (guard patterns are MQTT topic filters).
- **Retained messages = zero-bootstrapping state.** Agent cards and online/offline status are retained: a late-joining dashboard (or TUI) receives the whole fleet snapshot on its first SUBSCRIBE — no polling, no discovery service, no warm-up delay.
- **Last Will & Testament = free failure detection.** Each dispatcher registers its will at CONNECT; an unclean disconnect flips the retained status to “offline”. Crash detection is delegated to the protocol instead of heartbeat code.
- **Reactive, not polled.** The TUI and peers redraw on arrival of events only. Long LLM calls (20–90 s) stream per-request progress on `runes/prompts/req>/progress` — the UI stays live and other work keeps flowing.

How MQTT makes the harness perform better

- **Claim/lease turns broadcast into a work queue.** Every agent claims on `runes/prompts/req>/claim`, waits a 0.6 s window, and only the lexicographically-smallest claimant executes. Identical logic on every participant → exactly one winner, no central arbiter. Duplicates (retransmits, replays) are absorbed by TTL dedup + atomic execution reservation.
- **Conversational leases.** `goal/plan/mission` turns claim `runes/sessions/lease>/claim`; the lease id is derived identically on every agent (session id, or SHA-256 digest of the shared source) — sticky routing without state.
- **Correlation by topic, not by session table.** Replies land on `runes/prompts/req>/response` — the request id IS the reply address. Clients subscribe before publishing; no shared correlation store exists anywhere.
- **Agent-to-agent delegation.** A dispatcher addresses a peer directly at `runes/agents/id>/tasks`; results return on the requester’s reply topic. Distributed execution is just another publish.
- **Observability for free.** Wildcards: subscribe `runes/#` and you have a full fleet tap; `runes/_log/prompts(+ /latest retained)` feeds the durable JSONL journal that `bin/runes-replay` can show or replay.
- **Embedded, not operated.** The broker is an in-process Ruby class (~550 lines): QoS 0 fan-out (+ QoS 1/2 acks), retained, LWT, wildcards, UNSUBSCRIBE, per-socket write serialization, packet/retained/connection caps, time-boxed body reads. Dev fabric on localhost — external brokers (mosquitto) work unchanged.

Topic map

```
runes/prompts
  broadcast prompt (JSON envelope)
runes/prompts/<req>/claim
  build lease (claim race)
runes/prompts/<req>/progress
  per-request event stream
runes/prompts/<req>/response
  correlated reply
runes/sessions/<lease>/claim
  goal/plan/mission lease
runes/agents/<id>/card | status
  retained discovery / LWT presence
runes/agents/<id>/tasks[/<req>/response]
  delegation + correlated result
runes/tools/<tool>/request | response | error
  direct tool RPC (guard-gated)
runes/_log/prompts(+ /latest)
  journal feed (+ retained snapshot)
```

The bus is also the policy boundary

Tool capabilities are expressed as MQTT topic patterns and enforced by a default-deny guard: a manifest tool may publish only on `runes/tools/<tool>/response`; direct tool RPCs are matched against the same filter semantics (+ one level, # suffix). Merging is additive-only for builtins; malformed policies fail closed. The messaging model and the security model are the same model.

Engineering numbers

202 tests / 532 assertions green · claim window 0.6 s · identical-prompt TTL 60 s · packet cap 1 MiB · retained cap 1 000 entries · 256 connections · 64 subscriptions/client · LLM retry budget $\leq 1.5 \times$ timeout · WASM wall-clock 30 s (epoch interruption) + 5B fuel · 10 MiB journal rotation.